

DEVELOPMENT PARTNER BRIEF — FOR QUOTATION

Phase 1 Development Brief

Post-match highlight reel and court heatmap delivery — complete scope of work for development partner, with division of responsibilities, integration contracts, and acceptance criteria per component.

PREPARED BY

Manoj Maheshwari · PadelMind

PHASE

Phase 1 — Highlights + Heatmap (no shot classifier)

TARGET DELIVERY

12 weeks from engagement

AUTHORED

2026-07-09

PadelMind: Post-Match Highlight & Heatmap Delivery System

Purpose of this document: We are building an AI-assisted padel match analysis platform. This brief defines the exact scope we need a development partner to quote on for Phase 1. It also clearly states what our team will build and integrate at our end, so there is no confusion about ownership, interface contracts, or overlap.

Camera configuration locked — Plan A: Phase 1 uses a single IP camera per court, mounted 4–5 metres high behind one back glass wall. This is the deliberate choice for the pilot. Dual-camera (two-Pi, position-merge) is a Phase 2 upgrade once the first club is paying. Do not design or quote for multi-camera support in this phase.

1. What We Are Building — The Plain-Language Version

A padel player finishes their match at 7:30pm. At 7:42pm, their WhatsApp receives three messages automatically:

- 1. A **text summary** — how long the match was, how many rallies, which zones of the court they covered most.
- 2. A **heatmap image** — a top-down court diagram with a heat overlay showing where they spent each rally.
- 3. A **60-second highlight reel** — the top five rallies from the match, auto-clipped and stitched.

No app to download. No login during the match. The club staff sends one WhatsApp command when the match starts and one when it ends. The rest is automatic.

This is Phase 1. There is no shot classification, no skill scoring, and no drill recommendations in Phase 1. Those are Phase 2 features that depend on having a domain-expert coach on the team. Phase 1’s job is to prove the “wow moment” and sign the first paying club.

2. Full System Architecture

The system has six layers. The table below shows who owns each layer — **us (PadelMind team)** or **you (development partner)**.

LAYER	WHAT IT DOES	OWNER
Edge box (Raspberry Pi 5)	Records match from IP camera; detects match start/end; uploads MP4 to cloud storage	You (Sasank)
Cloud storage (Cloudflare R2)	Stores match video; triggers processing job	Us
Job orchestration (Cloudflare Worker + Upstash Redis)	Receives webhook from R2; queues job for GPU worker	Us
GPU inference worker (RunPod)	Downloads match; runs player tracking; produces positions JSON, heatmap PNG, highlight MP4	You (Sasank)
Database (Supabase)	Stores match metadata, player identities, heatmap URLs, highlight URLs	Us
WhatsApp delivery (AX Meta Cloud API)	Sends heatmap + highlight + summary to all 4 players post-match	Us

The handshake between your work and ours:

- Your Pi daemon → uploads `match.mp4` to our R2 bucket → fires our webhook
- Our Worker → queues a job → your RunPod worker picks it up
- Your RunPod worker → writes `heatmap_p1.png`, `heatmap_p2.png`, `heatmap_p3.png`, `heatmap_p4.png`, `highlights.mp4` back to our R2 → calls our webhook with the output URLs
- Our Worker → reads those URLs → sends the WhatsApp messages via our existing delivery infrastructure

You do not touch Supabase, the WhatsApp API, Cloudflare Workers, or the React PWA. We do not touch the Pi, the GPU inference code, or the media generation pipeline. Clean boundary.

3. What Our Team Brings (Not In Your Scope)

We are not starting from zero. The following is already built and running in production on other products we operate:

3.1 WhatsApp Delivery Infrastructure

We operate a WhatsApp Cloud API router (Meta BSP, our own WABA) that handles outbound messages including text, images, and video files. It currently serves a live B2B SaaS product with multiple tenants. Adding a `padel-match-report` message type to this router is a 2–3 hour integration on our end — not new infrastructure.

3.2 Cloudflare R2 + Workers

We use Cloudflare R2 as our primary object store and Cloudflare Workers for serverless edge logic on existing products. We will provision the R2 bucket, configure CORS, and write the job-dispatch Worker that receives your Pi's upload notification and queues the RunPod job.

3.3 Supabase Backend

We use Supabase (Postgres + Auth + Realtime) as our primary database and auth layer. We will design and own the schema:

```

padel_clubs      - club name, city, WhatsApp number, camera config
padel_courts    - court name, club reference, homography keypoints (JSON)
padel_matches   - match start/end, R2 video URL, processing status
padel_players   - player WhatsApp number, display name, club
padel_match_players - join table: which players played in which match;
                  stores heatmap URL + highlight URL per player

```

We own all reads and writes to these tables. Your RunPod worker will POST a JSON payload to one of our Cloudflare Worker endpoints when processing is complete — it does not need database credentials.

3.4 Player PWA (Progressive Web App)

We will build a mobile-first web application where players can review all their past matches, see full heatmap history, re-watch highlights, and share a stats card. Authentication is phone OTP (Supabase). The link sent in every WhatsApp message opens this PWA on the player's phone. This replaces any need for an App Store app. We build this in React on Cloudflare Pages.

3.5 ffmpeg — Media Foundation Knowledge

Our team has built ffmpeg-based video pipelines for other products (segment concat, trim, 9:16 crop, caption burn, BGM mix). We can advise on and review the ffmpeg commands in your highlight clipper, but you own the implementation and the rally-detection logic that drives it.

3.6 Club Registration Flow

The club sends one WhatsApp command to register a match:

```
PADEL START court1 +91XXXXXXXXXX +91XXXXXXXXXX +91XXXXXXXXXX +91XXXXXXXXXX
PADEL END court1
```

Our router parses this, creates the `padel_matches` row, and signals your Pi daemon to start/stop recording. We own this command parsing. You own the Pi daemon that listens for the signal.

4. What We Need From You (Your Scope)

There are six components in your scope. Each has its own acceptance criteria in Section 7.

Component 1 — Raspberry Pi Edge Software

What it does: The Pi (Model 5, 8GB RAM, 1TB NVMe) sits in a weatherproof enclosure at the club. It receives the RTSP stream from the Reolink RLC-823A IP camera (4K H.265), segments it into 10-minute MP4 chunks, detects when a match ends, concatenates the full match video, and uploads it to our R2 bucket.

Inputs: RTSP stream from camera (local network); a trigger signal from our system (HTTP POST or file-write) indicating match start/end.

Outputs: One concatenated `match_{id}.mp4` file uploaded to our R2 bucket; an HTTP POST to our Cloudflare Worker webhook confirming the upload is complete.

Key technical points:

- **MediaMTX** (formerly rtsp-simple-server) is the RTSP relay and segment recorder. Configuration is a single YAML file. We can provide a template.
- **Motion-based end detection** is the Phase 1 shortcut: a Python watchdog monitors the latest chunk for mean inter-frame pixel difference. When activity falls below threshold for 10 consecutive minutes after a live-match period, the match is considered over.
- **Primary trigger** for Phase 1: our system sends an HTTP POST to the Pi (`/match/start` , `/match/stop`) when the club sends the WhatsApp command. Motion detection is the fallback.
- Upload via `boto3` (Cloudflare R2 is S3-compatible). Credentials stored in `.env` on the Pi.
- The Pi must handle a reboot cleanly — daemon runs as a `systemd` service, auto-restarts.
- **4G/5G modem** (Jio or Airtel SIM) provides uplink. We handle SIM provisioning and modem hardware. You configure the Pi networking.

Dependencies we provide: - R2 bucket name, access key, secret key - Webhook URL to call on upload completion - Pi hardware (we procure; you configure)

Component 2 — GPU Inference Worker (RunPod)

What it does: A Docker container that runs on a RunPod RTX 3090 instance. It is invoked by our job queue, downloads the match video from R2, runs the computer vision pipeline, and produces structured position data for all four players plus a summary JSON.

Base library: `padel_analytics` (open-source, MIT licence). Repository: `github.com/Joao-M-Silva/padel_analytics`. You fork this, strip the Streamlit UI, and adapt it to run headlessly from the command line.

Pipeline stages your worker runs:

- 1. Court keypoints load** — read the 12 stored keypoint pixel coordinates from the JSON payload we include in the job. These were captured at camera install time using the calibration tool (Component 6 below). No re-clicking at runtime.
- 2. Court homography compute** — from the 12 keypoints, compute the perspective transform matrix that maps pixel positions to real-world court metres. This is already implemented in `padel_analytics/analytics/projected_court.py`.
- 3. Player detection + tracking** — YOLOv8m (person detection) + ByteTrack (multi-object tracking across frames). Already in `padel_analytics/trackers/players_tracker/`. Run at 1080p30 (downsample the 4K source before inference — saves 75% GPU time with negligible accuracy loss).
- 4. Position projection** — for every frame, for each tracked player: use the homography matrix to convert the player's `feet` pixel position to court metres (x, y). The `Player.projection` attribute in `players_tracker.py` already does this when a `ProjectedCourt` object is passed.
- 5. Output** — one JSON file per match: a list of frames, each containing player ID, court position (x_metres, y_metres), and frame timestamp. This is the input to Components 3 and 4.

Runtime target: 90-minute match processed in under 6 minutes on an RTX 3090.

Docker image requirements: - CUDA 12.x base - Python 3.11 - `ultralytics` (YOLOv8), `supervision`, `opencv-python-headless`, `numpy`, `scipy` - `boto3` for R2 access - Image must pull cold and be ready to process in under 90 seconds

Job interface (we call you):

We POST this JSON to your RunPod endpoint via HTTP:

```
{
  "job_id": "match_abc123",
  "video_r2_url": "https://r2.padelmind.com/matches/match_abc123.mp4",
  "court_keypoints": {
    "k1": [120, 980], "k2": [1800, 980],
    "k3": [120, 800], "k4": [960, 800], "k5": [1800, 800],
    "k6": [120, 540], "k7": [1800, 540],
    "k8": [120, 280], "k9": [960, 280], "k10": [1800, 280],
    "k11": [120, 100], "k12": [1800, 100]
  },
  "match_id": "match_abc123",
  "callback_url": "https://workers.padelmind.com/inference-complete"
}
```

Your callback to us (when done):

```
{
  "job_id": "match_abc123",
  "status": "success",
  "positions_r2_url": "https://r2.padelmind.com/inference/match_abc123_positions.json",
  "heatmap_urls": {
    "player_1": "https://r2.padelmind.com/inference/match_abc123_heatmap_p1.png",
    "player_2": "https://r2.padelmind.com/inference/match_abc123_heatmap_p2.png",
    "player_3": "https://r2.padelmind.com/inference/match_abc123_heatmap_p3.png",
    "player_4": "https://r2.padelmind.com/inference/match_abc123_heatmap_p4.png"
  },
  "highlight_r2_url": "https://r2.padelmind.com/inference/match_abc123_highlights.mp4",
  "stats": {
    "duration_minutes": 87,
    "rally_count": 12,
    "avg_rally_shots_est": 14
  }
}
```

Component 3 — Heatmap Renderer

What it does: Reads the positions JSON from Component 2 and renders one heatmap PNG per player. Each PNG is delivered to players via WhatsApp and displayed full-width in the PWA.

Technical specification:

- Input: positions JSON (one (x, y) in metres per frame, per player)
- Output: 4 PNG files, 800×1600px, dark background, fire/heat colour map
- Court dimensions: 10m × 20m (standard padel court)
- Library: `matplotlib` with `scipy.ndimage.gaussian_filter` for smooth heat distribution
- Court line overlay: baseline (y=0, y=20), service lines (y=3, y=17), net (y=10, dashed), side-wall lines (x=0, x=10)
- Player label at top of image (e.g. "Player 1 — Left Side")
- White text on dark background. Looks premium on a phone screen.
- File size: under 300KB per PNG (WhatsApp media limit compliance)

The heatmap generation is part of the RunPod worker Docker image (runs inline after tracking, before the callback). It does not require a separate service.

Component 4 — Rally Detector

What it does: Given the positions JSON, identifies the time windows when a rally was in progress. A rally is any continuous period where at least 2 players are visible AND their average movement velocity exceeds a threshold.

Algorithm (suggested — you may refine):

```
For each frame:
  - Compute per-player velocity: |position[t] - position[t-5]| / (5/fps)
  - If >= 2 players visible AND avg_velocity > 0.15 m/s: mark as RALLY
  - If < 2 players visible OR avg_velocity < 0.05 m/s: mark as REST
Apply 3-second minimum rally duration filter
Apply 30-second maximum gap merger (brief rests between shots in same rally)
Output: list of (start_frame, end_frame, duration_seconds) sorted by duration descending
```

Output: Top 8 rally windows by duration, embedded in the positions JSON. These are consumed by Component 5.

Edge cases to handle: - Players walking off court between games (not a rally) - One player retrieving a ball alone at the wall (not a rally) - Partial player occlusion reducing detected count to 3 temporarily

Component 5 — ffmpeg Highlight Clipper

What it does: Takes the top 5 rally windows from Component 4, clips those segments from the original match video, adds a 1-second buffer before and after each clip, and concatenates them into a single highlight reel.

Technical specification:

- Input: `match_{id}.mp4` (from R2) + rally window list from Component 4
- Output: `highlights.mp4` — a single continuous MP4
- Resolution: 1080×1920 (9:16 vertical) — crop from the 4K source
- Duration: 60–90 seconds total (5 clips × 12–18 seconds average)
- File size: under 15MB (WhatsApp video limit is 16MB)
- Codec: H.264 (libx264), AAC audio, CRF 26–28, preset `fast`
- Between clips: 0.3-second black frame fade

ffmpeg approach (per clip):

```
ffmpeg -ss {start_s} -i match.mp4 -t {duration_s + 2} \
-vf "scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920,
    fade=t=in:st=0:d=0.2,fade=t=out:st={duration_s}:d=0.2" \
-c:v libx264 -crf 27 -preset fast -c:a aac -b:a 128k \
clip_{n}.mp4
```

Then concatenate with `ffmpeg -f concat -safe 0 -i clips.txt`.

The video crop should favour the half of the court where the most action occurred (use position data to determine dominant zone). This is a nice-to-have, not a blocker.

Component 6 — Court Calibration Tool

What it does: A lightweight browser-based tool used once at camera install time. The installer opens this page, uploads a still frame from the camera, and clicks on 12 predefined court keypoints. The tool stores the pixel coordinates to our Supabase database under the relevant `padel_courts` record. These coordinates are then included in every job payload so the GPU worker knows the court geometry without re-clicking.

The 12 keypoints (standard padel court layout):



Technical specification:

- Simple HTML page with `<canvas>` overlay on the uploaded frame
- Click to place each of 12 numbered markers in sequence
- Markers draggable to refine position
- “Save” button POSTs `{ court_id, keypoints: { k1: [x,y], ... k12: [x,y] } }` to our Cloudflare Worker endpoint, which writes to Supabase
- No auth required (tool is used once per court, behind a private URL)
- Works on laptop browser only (no mobile requirement)
- Stack: vanilla HTML + JS — no framework needed

This is the simplest component in your scope. ~200 lines of vanilla JS.

5. Integration Contract Summary

SIGNAL	FROM	TO	FORMAT
Match start trigger	Our WA router	Pi daemon	HTTP POST <code>{ match_id, court_id }</code> to Pi local IP
Match end trigger	Our WA router	Pi daemon	HTTP POST <code>{ match_id }</code> to Pi local IP
Match MP4 upload complete	Pi daemon	Our Cloudflare Worker	HTTP POST <code>{ match_id, r2_url }</code>
Job dispatch	Our Cloudflare Worker	RunPod endpoint	HTTP POST (see Section 4.2 job interface)
Inference complete	RunPod worker	Our Cloudflare Worker	HTTP POST (see Section 4.2 callback)
Keypoints save	Calibration tool	Our Cloudflare Worker	HTTP POST <code>{ court_id, keypoints }</code>

No direct database access is needed by any of your components. All persistence goes through our Cloudflare Worker endpoints, which you call via HTTP. We provide those endpoint URLs before development begins.

6. Infrastructure & Runtime Environment

RESOURCE	SPEC	PROVISIONED BY
IP Camera	Reolink RLC-823A 4K PoE	Us
Raspberry Pi 5	8GB RAM, 1TB NVMe, IP66 enclosure	Us
4G/5G modem	Jio/Airtel SIM, unlimited data	Us
R2 bucket	Cloudflare R2, <code>padelmind-dev</code>	Us
GPU worker endpoint	RunPod RTX 3090, on-demand	Us (we provision; you build the Docker image)
Redis queue	Upstash free tier	Us
Docker registry	GitHub Container Registry	You (push image; we pull)
Supabase database	<code>padelmind</code> project	Us

We will provide you with: - R2 bucket credentials (access key + secret for dev and prod buckets) - RunPod API key + endpoint ID - Webhook URLs for each integration point - A test match video (20 minutes, real padel footage) for integration testing - A reference court keypoints JSON for the test video

7. Acceptance Criteria — Per Component

Each component is accepted when it passes the following test on the supplied test video.

COMPONENT	ACCEPTANCE TEST
Pi edge software	Match MP4 appears in R2 within 3 minutes of receiving the <code>/match/stop</code> signal; file is playable; size is within 10% of expected for match duration
GPU inference worker	Processes 90-minute match in under 6 minutes on RTX 3090; positions JSON has entries for at least 3 of 4 players in >70% of frames; court coordinates are within the 10m×20m boundary
Heatmap renderer	4 PNG files generated; each under 300KB; court lines clearly visible; heat zones correspond visually to where players spent time (verified by watching the video)
Rally detector	Identifies at least 5 distinct rally windows in a 90-minute match; fewer than 3 false positives (walking, single-player warmup) in the top-8 output
Highlight clipper	Produces a single MP4 under 15MB; 5 clips visible; clips correspond to the rally windows; video is watchable on an Android WhatsApp at 1080p
Calibration tool	Clicking 12 points on a test frame and saving results in correct JSON in the database; the same keypoints used in the GPU job produce a valid homography (player walks from baseline to net, position in output JSON follows expected trajectory)

8. What Is Explicitly Out of Scope for This Phase

The following will not be built in Phase 1 and should not be quoted:

- Shot classification (forehand, backhand, smash, bandeja, etc.) — requires trained model, Phase 2
- Quality scoring or coaching feedback — requires domain coach input, Phase 2
- Ball tracking (TrackNet) — requires retraining for padel ball, Phase 2
- Player skill score or rating — requires classifier, Phase 2
- Player-facing mobile app (iOS / Android native) — we ship a PWA instead
- Live streaming or real-time analysis — Phase 3
- Multi-camera per court — Phase 3
- Booking system integration (Hudle, Playo) — Phase 2

9. Questions You Will Likely Have

- Q: Which YOLOv8 model size?** YOLOv8m (medium). Large is more accurate but doesn't fit within the 6-minute processing budget on RTX 3090 for a 90-minute match. If you find accuracy is insufficient at medium, raise it — we'll discuss budget impact.
- Q: The padel_analytics repo has a Streamlit UI. Do I need to preserve it?** No. Fork the repo, strip the UI entirely. You need only the tracker classes and analytics modules. The UI code is irrelevant to the headless Docker worker.
- Q: What if fewer than 4 players are detected in some frames?** That is expected — players go near walls, get partially occluded. Minimum 2 detected players is sufficient to determine a rally is happening. The heatmap for a player with sparse data will just show a more concentrated heat zone rather than a full-court spread.
- Q: Can the Pi handle 4K RTSP decoding?** No, and it doesn't need to. MediaMTX relays and segments the stream without decoding it — pure bitstream passthrough to file. The Pi never decodes frames. Decoding happens on the RunPod GPU.

Q: What is the network bandwidth requirement for the Pi upload? A 90-minute 4K H.265 match is approximately 3–4 GB. On a Jio 5G SIM with ~50 Mbps uplink, that is about 10 minutes upload time. The Pi should begin upload as soon as the match ends — the RunPod job is queued but waits for upload to complete before starting.

Q: Do I need to handle multiple cameras per court? No. Phase 1 is one camera per court. The architecture supports multiple cameras later but you should not design for it now.

10. Summary — Division of Work at a Glance

WHAT SASANK BUILDS	WHAT PADELMIND TEAM BUILDS
Pi recording daemon + upload	R2 bucket + Cloudflare Worker job dispatch
GPU inference worker (Docker)	Upstash queue management
Player tracking pipeline	Supabase schema + all DB operations
Heatmap PNG renderer	WhatsApp delivery (all 4 players)
Rally detector algorithm	Player PWA (React, all match history)
ffmpeg highlight clipper	Club registration WA command parser
Court calibration web tool	Infrastructure provisioning (RunPod, R2, Supabase)
Docker image on GHCR	Monitoring + alerting

11. Phase 1 Camera Hardware — Plan A Specification

Single camera per court. No multi-camera in this phase.

ITEM	SPEC	APPROX. INR
IP Camera	Reolink RLC-823A, 4K H.265, PoE, IP66 weatherproof	₹12,000–18,000
PoE Switch	TP-Link TL-SG1005P (5-port, 65W total)	₹3,000
Raspberry Pi 5	8GB RAM + official active cooler	₹8,500
NVMe SSD	1TB WD Green M.2 2242 + Pi 5 NVMe HAT	₹5,500
4G/5G Modem	Jio or Airtel USB dongle + unlimited SIM	₹3,000 hardware + ₹1,000/mo
Weatherproof enclosure	IP65 DIN box, 200×160×75mm	₹1,500
Cat6 cable + PoE run	20m run from switch to camera mount point	₹1,500
Camera mount	Heavy-duty articulating wall bracket	₹800
Total per court (hardware only)		₹36,000–42,000

12. Effort Matrix — Your Estimate, With Our Technical Context

How to read this section

We have done a full technical review of every component in your scope — including reading the open-source `padel_analytics` repository that forms the core of the GPU inference work. The table below describes each task, its complexity, and the specific technical reasons we think it is the way it is.

We are not giving you our day estimates. We want your honest assessment of how long each task will take you, given how you work and what tools you use.

One assumption we are making explicitly: We expect you to use AI coding tools — Claude Code, Cursor, Copilot, or equivalent — to accelerate the implementation. This is not a criticism; it is a baseline expectation. Code generation tools significantly compress effort on well-defined tasks. Please factor that into your estimate and include any AI tool subscription costs (monthly cost of whatever you use) as a line item in your quote. We will cover that cost as part of the engagement.

What we are asking from you:

For each task, give us your estimate in days. If your technical reading of the task differs from the reasoning we have stated, tell us why — with a verifiable fact. We have done our homework on this architecture and we will check any claim you make. A well-reasoned challenge to our technical framing is valuable. An inflated number without explanation is not.

Complexity key: **L** = Low (standard tooling, well-understood problem) · **M** = Medium (design decisions, some unknowns) · **H** = High (architectural risk, external dependencies, potential rework)

Component 1 — Raspberry Pi Edge Software

Context: This is plumbing. MediaMTX is a zero-config binary. The match daemon is a short Python script. The integration test takes the most clock-time because it requires a live camera — but the code itself is simple.

TASK	COMPLEXITY	OUR TECHNICAL NOTES	YOUR ESTIMATE (DAYS)
MediaMTX install + RTSP relay + segment config	L	Single YAML file; binary runs as a systemd service; no custom code required	
HTTP endpoint on Pi: <code>/match/start</code> , <code>/match/stop</code>	L	Flask or FastAPI; ~30 lines; stores state to a local file	
Chunk detection + ffmpeg concat on match end	M	Scan segment directory; <code>ffmpeg -f concat</code> ; handle the partial last chunk	
Motion-based fallback end detection	M	Mean inter-frame pixel diff on latest chunk; two configurable thresholds (live / idle)	
R2 upload via boto3 + retry with backoff	L	Cloudflare R2 is S3-compatible; standard <code>upload_file</code> with exponential backoff on 5xx	
Webhook POST to our Worker on upload complete	L	Single HTTP POST; payload format pre-specified (Section 5)	
systemd service + auto-restart on reboot	L	Standard unit file; <code>Restart=always</code> ; <code>WantedBy=multi-user.target</code>	
Integration test: camera → chunk → concat → R2 playable	M	Requires live camera; verify MP4 lands in R2 and plays cleanly	
Component 1 Total			

Component 2 — GPU Inference Worker (RunPod)

Context: This is the heaviest component. The `padel_analytics` repo (MIT licence) already has YOLOv8 player tracking, ByteTrack, and court homography implemented — but the codebase is Streamlit-first and batch-style. Your job is to strip the UI, containerise it for RunPod, wire it to R2, and hit the performance target. The CUDA environment setup and performance tuning are where the real risk lives.

TASK	COMPLEXITY	OUR TECHNICAL NOTES	YOUR ESTIMATE (DAYS)
Fork <code>padel_analytics</code> ; strip <code>app.py</code> + <code>ui.py</code> ; add <code>worker.py</code> headless entry	L	No logic change; pure file deletion + new entry point	
Docker image: CUDA 12 base + ultralytics + supervision + boto3	M	Layer ordering matters for image size; CUDA base images from nvcr.io are known; first build is slow	
RunPod serverless HTTP endpoint: receive job JSON, parse	L	RunPod serverless handler pattern is documented; straightforward wrapper	
R2 video download at job start	L	<code>boto3.download_file</code> ; 3–4 GB takes 2–4 min on RunPod network — plan for this in timeout config	
Court keypoints JSON → homography matrix	L	<code>ProjectedCourtKeypoints</code> + <code>ProjectedCourt</code> already implemented in <code>analytics/projected_court.py</code> ; pass keypoints in, get matrix out	
YOLOv8m + ByteTrack player tracking (headless)	M	<code>PlayersTracker</code> already written in <code>trackers/players_tracker/</code> ; remove UI drawing calls, add frame-stride option to meet perf target	
Player position projection to court metres per frame	M	<code>Player.projection</code> is already computed when a <code>ProjectedCourt</code> object is passed in; wire it up and serialize per frame	
Output: positions JSON (all players, all frames) → R2	L	Standard <code>json.dumps</code> + <code>boto3.upload_fileobj</code>	
R2 upload all outputs: positions JSON + 4 PNGs + highlight MP4	L	Same boto3 pattern as Pi upload; 5 files in sequence	
Callback POST to our webhook with all output URLs	L	Single HTTP POST; payload format pre-specified (Section 5)	
Performance tuning: 90-min match processed in under 6 min on RTX 3090	H	Profile frame-stride (every 3rd frame is a known approach); verify tracking accuracy holds; GPU utilisation check with <code>nvidia-smi</code>	
Integration test on supplied test video + keypoints	M	We provide a 20-min match MP4 and court keypoints JSON; verify positions file, 4 PNGs, and highlight MP4 arrive in R2	
Component 2 Total			

Component 3 — Heatmap Renderer

Context: The position data from Component 2 gives you (x, y) in court metres for every player, every frame. The heatmap is a gaussian density render over a court diagram. Standard matplotlib + scipy. The output must look good on a phone screen — dark background, fire colourmap, clear court lines.

TASK	COMPLEXITY	OUR TECHNICAL NOTES	YOUR ESTIMATE (DAYS)
Court diagram base: matplotlib figure with all line markings	L	6 horizontal lines + 2 side verticals; court is 10m wide × 20m long; fixed	
Gaussian heatmap from positions: <code>np.histogram2d</code> + <code>gaussian_filter</code>	L	Standard density estimation; sigma tuning for visual spread	
4-player PNG generation: dark background, fire colourmap, 800×1600px	L	Loop over 4 player IDs; <code>plt.savefig</code> per player	
Player label + court-side annotation (Left / Right half)	L	<code>ax.set_title</code> ; side derived from player's average x-position	
File size under 300KB per PNG	L	<code>dpi=120</code> + <code>optimize=True</code> on save; test on a real output	
Inline in RunPod worker — not a separate service	L	Function call after tracking loop; no new infrastructure	
Integration test: hot-zones match where player actually stood	L	Visual comparison against test video	
Component 3 Total			

Component 4 — Rally Detector

Context: No ML. Pure algorithmic work on the positions JSON from Component 2. The state machine is straightforward; the effort is in threshold calibration against real footage. That calibration loop is the hardest part to compress with AI tooling — it requires watching video and iterating.

TASK	COMPLEXITY	OUR TECHNICAL NOTES	YOUR ESTIMATE (DAYS)
Per-player per-frame velocity: <code> pos[t] - pos[t-5] / (5/fps)</code>	M	Sliding window; handle frames where player is not detected	
Rally state machine: LIVE / REST transitions with hysteresis	M	Two configurable floats (live_threshold, rest_threshold); hysteresis prevents flicker between states	
Minimum rally duration filter: discard windows < 3 seconds	L	Single filter pass on state machine output	
Short-gap merger: join windows separated by < 30 seconds	M	Prevents one long rally being split by a brief ball retrieval	
False positive suppression: fewer than 2 players detected = not a rally	M	Count players per frame window; apply before state machine	
Output: top-8 rally windows by duration, appended to positions JSON	L	Sort by duration descending; slice top 8	
Integration test: manually verify top-8 against test video	M	Watch the video; count false positives (walking, single-player warm-up); thresholds likely need 2–3 iteration passes	
Component 4 Total			

Component 5 — ffmpeg Highlight Clipper

Context: We use ffmpeg ourselves for reel stitching and media pipelines, so we know this territory. The work is straightforward. The one non-trivial constraint is hitting under 15MB reliably across different match footage without visible degradation — CRF tuning varies by content.

TASK	COMPLEXITY	OUR TECHNICAL NOTES	YOUR ESTIMATE (DAYS)
Per-rally clip extraction with 1-second padding	L	<code>ffmpeg -ss {start-1} -t {dur+2} -c copy</code> — stream copy, no re-encode	
9:16 vertical crop (1080×1920) from 4K source	L	<code>scale=1080:1920:force_original_aspect_ratio=increase,crop=1080:1920</code> ; crop centred	
Fade in/out: 0.2 seconds at head and tail of each clip	L	<code>fade=t=in:st=0:d=0.2</code> and <code>fade=t=out</code> ; standard parameter	
Multi-clip concat to single MP4	L	ffmpeg concat demuxer with a clips list file	
File size under 15MB: CRF 26–28 + preset fast	M	Content density affects bitrate; test on real match footage; may need 2-pass	
Inline in RunPod worker after Component 4 completes	L	Sequential function call; no new infrastructure	
Integration test: MP4 under 15MB, watchable, WhatsApp-deliverable	L	Send to our WA test number; verify receipt and full playback	
Component 5 Total			

Component 6 — Court Calibration Tool

Context: A single HTML page used once per court at camera install time. No framework, no backend. The installer uploads a still frame from the camera and clicks 12 court keypoints in sequence. Draggable refinement is standard canvas mouse-event code. The Save button fires one HTTP POST to our endpoint.

TASK	COMPLEXITY	OUR TECHNICAL NOTES	YOUR ESTIMATE (DAYS)
HTML page: file upload + canvas overlay on uploaded frame	L	<code>FileReader</code> API → <code>canvas.drawImage</code> ; single self-contained HTML file	
Sequential click-to-place: 12 numbered markers in order	M	Canvas click handler; marker index auto-advances; draw marker circle at click position	
Draggable marker refinement	M	<code>mousedown</code> → hit-test nearest marker → <code>mousemove</code> → update position → <code>mouseup</code>	
Reference diagram sidebar: court layout with k1-k12 labels	L	Static SVG showing which point to click next; prevents install errors	
Save: POST <code>{ court_id, keypoints }</code> to our endpoint	L	Single <code>fetch</code> POST; we provide the URL and expected JSON shape	
Integration test: saved keypoints produce valid homography in worker	M	We run the worker with the saved keypoints on the test video; verify player positions land within court bounds	
Component 6 Total			

End-to-End Integration

Context: Once all six components pass their own acceptance tests, the full pipeline needs a joint smoke test end-to-end. This is where the boundary bugs always appear — timing mismatches between the Pi upload and the job queue, callback payload format mismatches, WhatsApp file-size rejections. The bug-fix cycle here is real.

TASK	COMPLEXITY	OUR TECHNICAL NOTES	YOUR ESTIMATE (DAYS)
Joint smoke test: real match, full pipeline, WA messages delivered	H	First live run across the Pi → R2 → RunPod → WA boundary; timing and format issues are expected	
Bug-fix cycle post smoke test	H	Most issues surface at the integration boundaries, not inside individual components	
README per component + complete environment variable list	L	Required for our ongoing ops; must cover every config knob	
Handover + knowledge transfer call	L	1-hour recorded video call; walkthrough of each component	
Integration Total			

Effort Summary — Your Numbers

COMPONENT	YOUR ESTIMATE (DAYS)
1 — Pi Edge Software	
2 — GPU Inference Worker	
3 — Heatmap Renderer	
4 — Rally Detector	
5 — ffmpeg Highlight Clipper	
6 — Court Calibration Tool	
End-to-End Integration	
Total	

What We Need Back From You

Step 1 — Effort estimates. Fill in your day estimates in the tables above. Where your reading of a task differs technically from what we have written, flag it — with the specific technical reason. We will verify any factual claim.

Step 2 — Overall workability verdict. After reviewing the full system architecture (Sections 2–10) and the task breakdown above:

- Is the overall plan technically sound as designed?
- Are there any architectural decisions you would approach differently, and why?
- Are there any risks we have not identified that you consider real?

We want your honest technical opinion. If you see something wrong with our approach, say so before build — not after.

Step 3 — Cost estimate. Based on your effort estimates above:

1. Your **rate** — daily rate or fixed-price per component, in INR
2. Your **AI tooling costs** — monthly subscription cost of any AI coding tools you use (Claude Code, Cursor, Copilot, etc.); list the tool and the cost; we cover this as a project expense
3. Your **total cost** in INR
4. Your **payment milestones** — we prefer milestone-based payment tied to the acceptance tests in Section 7, not time-based
5. Your **earliest start date**
6. Any **dependencies on us** before you can begin — test video, R2 credentials, Pi hardware access, etc.

Target: A working end-to-end pipeline at one pilot court within **12 weeks of engagement start**.

Companion document set: Strategic Venture Plan · Project Report · Technical Feasibility · Commercial Report · Base Club Offer · Technical Review Request.

Single source of truth maintained in `~/Documents/padel-clone/`.